Contents

Dynamics of a Magnetically-Driven Spinning Micro Flying Robot 1
0. System description and modeling assumptions . . . 1
1. Notation and unit conventions . . . 2
2. Rotational kinematics: the phase variable . . . 3
3. Moment of inertia . . . 3
3.1 The two-rod approximation — when is it valid? . . . 4
4. Torques on the robot . . . 4
4.1 Magnetic drive torque . . . 4
4.2 Aerodynamic drag torque — blade-element derivation . . . 5
4.3 Fitting k_d from data — through-origin least squares . . . 5
5. Phase (synchronization) dynamics . . . 6
5.1 The two-state model . . . 6
5.2 The swing equation . . . 6
5.3 Equilibria and their stability . . . 6
5.4 Step-out ceiling . . . 7
5.5 Small-signal behavior: natural frequency and damping . . . 7
5.6 Ramp feasibility (quasi-static torque budget) . . . 7
6. Vertical (heave) dynamics . . . 7
6.1 Blade-element thrust . . . 7
6.2 Vertical equation of motion and hover . . . 8
6.3 Heave damping from inflow . . . 8
6.4 Linearization about hover: frequency → altitude . . . 8
7. The coupled nonlinear model . . . 9
8. Linearized state-space model . . . 9
8.1 Trim point . . . 9
8.2 Jacobians . . . 9
8.3 Eigenvalues (modes) . . . 10
8.4 Input–output transfer functions . . . 10
8.5 Structural properties . . . 11
9. Parameter summary (values from the code / defaults) . . . 11
10. Limitations and extensions . . . 11
Appendix A — Correspondence with the MATLAB implementation . . . 12
Appendix B — Why $\dot{f}_{max}$ and f_{max} are the two feasibility axes . . . 12

Dynamics of a Magnetically-Driven Spinning Micro Flying Robot

Complete lecture notes: physical modeling, derivations, and state-space formulation. Companion to `frequency_modulation_piecewise_polynomial_gui.m`.

---

0. System description and modeling assumptions

The robot is a millimeter-scale rotor: a magnetized central body carrying four blades. A **rotating external magnetic field** (generated by a coil array) drags the robot's in-plane magnetization around; the resulting spin drives the blades, which produce aerodynamic **thrust** (lift) and **drag torque**. Modulating the field's rotation frequency modulates spin speed, hence thrust, hence altitude.

Assumptions (A1–A6):

- **A1 — Coaxial alignment.** The robot's spin axis coincides with the field's rotation axis ($\hat{z}$). Justified by gyroscopic stiffness at 100–230 Hz spin rates and by the restoring out-of-plane magnetic torque, which makes the aligned state an attractor.
- **A2 — Rigid body.** Blades do not flex; inertia is constant.
- **A3 — Uniform field over the robot.** The coil array's working volume is Helmholtz-like: the field applies pure torque (no translational magnetic force, since force couples to $\nabla B \approx 0$), and the torque amplitude τ_{max} is position-independent.
- **A4 — Linear magnetics.** Air-core coils (or cores well below saturation): $B \propto I_{coil}$, so τ_{max} is a constant set by the drive amplitude.
- **A5 — Quadratic aerodynamics.** Blade-element Reynolds number is in the pressure-drag regime ($Re \sim$ 10-100), so local aerodynamic forces scale with the *square* of local airspeed. Validated empirically (§4.3, R^2 reported by the GUI).
- **A6 — Hover-referenced aerodynamic coefficients.** Drag and thrust coefficients are those measured/fitted near hover; inflow corrections for climb appear only as the first-order heave damping term (§6.3).

---

1. Notation and unit conventions

Symbol	Meaning	Units
$\theta_f,\ \theta_r$	field / robot angle in the rotation plane (from an arbitrary fixed lab direction)	rad
$\delta = \theta_f - \theta_r$	phase lag of robot behind field	rad
$\omega = \dot{\theta}_r$	robot angular velocity	rad/s
$f = \omega/2\pi$	robot spin frequency	Hz
$f_f(t)$	commanded field rotation frequency (**the control input**)	Hz
I	robot moment of inertia about spin axis	kg·m²
$\tau_{max} = mB$	peak magnetic torque (dipole moment × field)	N·m
k_d	drag-torque coefficient ($\tau_{drag} = -k_d f^2$)	N·m/Hz²
k_T	thrust coefficient ($T = k_T f^2$)	N/Hz²
M	torque margin $= \tau_{max}/(k_d f_0^2)$ at start frequency f_0	–
$z,\ w = \dot{z}$	altitude, climb rate	m, m/s
$m_R,\ g$	robot mass, gravity	kg, m/s²

Radian/Hz convention. Physics (Newton's law, torques, the ODE state) lives in radians; commands, data fits, and plots live in Hz. Every factor of 2π marks a border crossing: $\omega = 2\pi f$, $\dot{\omega} = 2\pi \dot{f}$.

---

2. Rotational kinematics: the phase variable

Fix lab axes with $\hat{z}$ along the field's rotation axis and any reference direction $\hat{x}$ in the plane. The field vector and the robot's in-plane magnetization are

$$\mathbf{B} = B\,[\cos\theta_f,\ \sin\theta_f,\ 0], \qquad \mathbf{m} = m\,[\cos\theta_r,\ \sin\theta_r,\ 0].$$

Only the difference $\delta = \theta_f - \theta_r$ enters the dynamics (the reference direction is arbitrary), so the natural kinematic chain is

$$\delta = \theta_f - \theta_r, \qquad \dot{\delta} = 2\pi f_f - \omega, \qquad \ddot{\delta} = 2\pi \dot{f}_f - \dot{\omega}.$$

$\dot{\delta}$ is the rate at which the gap opens; $\ddot{\delta}$ is the *relative* angular acceleration (field minus robot). Working in δ is equivalent to riding in the field's rotating frame and watching the robot swing relative to you.

---

3. Moment of inertia

The robot is modeled as a central body plus four blades, each blade as a small cylinder (radius r_b, length h_b) whose center sits a distance d from the spin axis.

Cylinder about a transverse axis through its own center:

$$I_{cyl} = \tfrac{1}{12} m_b\,(3r_b^2 + h_b^2).$$

Parallel-axis shift to the spin axis:

$$I_{blade} = \tfrac{1}{12} m_b (3r_b^2 + h_b^2) + m_b d^2.$$

Total (body term I_0 plus four identical blades):

$$\boxed{I = I_0 + 4\left[\tfrac{1}{12} m_b (3r_b^2 + h_b^2) + m_b d^2\right]}$$

With the code's values ($I_0 = 3.89 \times 10^{-9}$ kg·m², $m_b = 1.17832 \times 10^{-5}$ kg, $r_b = h_b = 0.79375$ mm, $d = 0.496875$ mm — the 10^{-6} factors convert mm² → m²):

$$I = 3.89 \times 10^{-9} + 4(2.475 \times 10^{-12} + 2.909 \times 10^{-12}) = 3.912 \times 10^{-9}\ \mathrm{kg \cdot m^2}.$$

The central body dominates; all four blades together contribute only ≈ 0.55 %.

> **History note:** the MATLAB file originally multiplied the blade term by `2.0` ($I = 3.901 \times 10^{-9}$, ≈ 0.28 % low). It has been corrected to `n_blades = 4` and refactored into named parameters (`I_body`, `m_blade`, `r_blade`, `h_blade`, `d_blade`). The dynamic effect of the correction is negligible ($\omega_n \propto 1/\sqrt{I}$ shifts by ≈ 0.14 %).

3.1 The two-rod approximation — when is it valid?

Since the four blades form two diametrically-opposite pairs, it is tempting to model each pair as **one rod of tip-to-tip length $2b$ about its center**, $b = d + h_b/2$. Check this against the exact result. For a thin rod segment of density $\lambda = m_b/h_b$ occupying $r \in [a, b]$ with $a = d - h_b/2$:

$$I_{blade} = \int_a^b \lambda r^2\,dr = \frac{\lambda}{3}(b^3 - a^3) = \tfrac{1}{12}m_b h_b^2 + m_b d^2$$

— the direct integral reproduces the parallel-axis formula exactly. A true blade *pair* is therefore a rod of length $2h_b$, mass $2m_b$, **with the hub segment $[-a, a]$ missing**. Replacing it by a continuous full-span rod of the same mass,

$$I_{rod} = \tfrac{1}{12}(2m_b)(2b)^2 = \tfrac{2}{3}m_b b^2,$$

redistributes mass inward into the empty hub region and *underestimates* the pair's inertia. With this robot's geometry ($a = 0.098$ mm, $b = 0.895$ mm):

$$I_{rod} = 0.533\,m_b\ \text{mm}^2 \quad \text{vs.} \quad I_{pair} = 2\big[\tfrac{1}{12}m_b h_b^2 + m_b d^2\big] = 0.599\,m_b\ \text{mm}^2 \qquad (\approx 11\,\%\ \text{low}).$$

Exactness condition: the two-rod model is exact iff the blades root at the axis, $a = 0 \iff d = h_b/2$, where the pair literally is a continuous center rod: $\frac{1}{12}(2m_b)(2h_b)^2 = 2[\frac{1}{12}m_b h_b^2 + m_b(h_b/2)^2]$. This robot is close to that limit ($a \ll b$), which is why the error is only ~11 %.

Practical verdict: since the blade term is ≈ 0.55 % of total I, the 11 % error would shift I by only ≈ 0.06 % — numerically harmless. The per-blade parallel-axis form is kept anyway: it is exact for any hub radius and generalizes to other blade counts/offsets. Caveat: the code's blade element is a *cylinder*, $\frac{1}{12}m_b(3r_b^2 + h_b^2)$, and with $r_b = h_b$ the transverse term $3r_b^2$ dominates the rod term h_b^2; a thin-rod shortcut silently drops it. (If the physical blade is a thin flat plate of chord w, the correct element is $\frac{1}{12}m_b(w^2 + h_b^2)$ — worth checking against the CAD.)

---

4. Torques on the robot

4.1 Magnetic drive torque

A dipole in a field experiences $\tau = \mathbf{m} \times \mathbf{B}$. With both vectors in-plane (A1), the component along $\hat{z}$ is

$$\tau_z = mB\sin(\theta_f - \theta_r) = \tau_{max}\sin\delta.$$

Properties that shape the whole problem:

- **Restoring up to 90°:** more lag → more torque, but only until $\delta = \pi/2$ where torque peaks at τ_{max}. Beyond 90° torque *falls* with lag → loss of synchronization ("step-out", pole slipping) becomes possible.

- **Linear in coil current** (A4): the coil array obeys superposition, $\mathbf{B}(p) = A(p)\,\mathbf{I}_{coil}$, so $\tau_{max} \propto I_{coil}$. Distance enters through the field map — approximately constant inside the designed uniform region, $\propto 1/r^3$ (dipole far field) outside; never linear in r.

Margin parameterization. Instead of specifying B in tesla, define the margin at the initial frequency f_0:

$$\tau_{max} = M \cdot k_d f_0^2, \qquad M \geq 1.$$

M measures torque headroom over the drag that must be overcome to spin at f_0.

4.2 Aerodynamic drag torque — blade-element derivation

A blade element at radius r, chord $c(r)$, moves with local airspeed $v(r) = \omega r = 2\pi f\, r$. In the pressure-drag regime (A5), its drag force is

$$dF(r) = \tfrac{1}{2}\rho\, C_D\, c(r)\, v(r)^2\, dr,$$

and its torque contribution is $r\, dF$. Integrating over the blades:

$$\tau_{drag} = \tfrac{1}{2}\rho C_D (2\pi f)^2 \int_{blades} c(r)\, r^3\, dr \;\equiv\; k_d\, f^2.$$

All geometry and air properties collapse into k_d (units N·m/Hz² — the Hz² is f^2). In the dynamics we write $-k_d f\,|f|$ so the torque always opposes rotation.

4.3 Fitting k_d from data — through-origin least squares

Given measured drag torques τ_i at frequencies f_i (the code's 23 points, 10–230 Hz), fit the one-parameter model $\tau = -kf^2$ by minimizing

$$J(k) = \sum_i \left(\tau_i + k f_i^2\right)^2.$$

Setting $dJ/dk = 0$:

$$\sum_i 2 f_i^2 (\tau_i + k f_i^2) = 0 \quad \Longrightarrow \quad \boxed{k_d = \frac{\sum_i f_i^2\,(-\tau_i)}{\sum_i f_i^4}}$$

— exactly the `k_drag = sum(f2 .* (-drag_torque_points)) / sum(f2.^2)` line in the code. The coefficient of determination $R^2 = 1 - SS_{res}/SS_{tot}$ is the empirical check on the quadratic form: a near-unity R^2 justifies A5. With the file's data, $k_d \approx 3.91 \times 10^{-10}$ N·m/Hz².

---

5. Phase (synchronization) dynamics

5.1 The two-state model

Newton's second law for rotation plus the phase kinematics give the model the GUI integrates (the `rotationODE` anonymous function in `runSimulation`):

$$\boxed{\begin{aligned}\dot{\delta} &= 2\pi f_f(t) - \omega \\ I\,\dot{\omega} &= \tau_{max}\sin\delta \;-\; k_d\,\frac{\omega}{2\pi}\left|\frac{\omega}{2\pi}\right|\end{aligned}}$$

5.2 The swing equation

Eliminate $\omega = 2\pi f_f - \dot{\delta}$ using $\dot{\omega} = 2\pi \dot{f}_f - \ddot{\delta}$:

$$I(2\pi \dot{f}_f - \ddot{\delta}) = \tau_{max}\sin\delta - k_d\Big(f_f - \frac{\dot{\delta}}{2\pi}\Big)^2.$$

Expanding the drag term to first order in $\dot{\delta}$ (robot near command speed), $k_d(f_f - \dot{\delta}/2\pi)^2 \approx k_d f_f^2 - (k_d f_f/\pi)\,\dot{\delta}$, and rearranging:

$$\boxed{I\,\ddot{\delta} \;+\; \underbrace{\frac{k_d f_f}{\pi}}_{c\ \text{(damping)}}\;\dot{\delta} \;+\; \underbrace{\tau_{max}\sin\delta}_{\text{restoring}} \;=\; \underbrace{2\pi I \dot{f}_f}_{\text{inertial demand}} \;+\; \underbrace{k_d f_f^2}_{\text{drag load}}}$$

This is the **swing equation** — identical in form to a synchronous machine on a power grid or a torque-driven pendulum. The right-hand side is the torque the field must supply for perfect tracking; the robot "pays" for it by settling at whatever lag δ makes $\tau_{max}\sin\delta$ match the demand.

5.3 Equilibria and their stability

For constant f_f ($\dot{f}_f = 0$), equilibria satisfy

$$\sin\delta_{eq} = \frac{k_d f_f^2}{\tau_{max}} = \frac{1}{M}\Big(\frac{f_f}{f_0}\Big)^2 \;\equiv\; s_e \in (0,1].$$

Two solutions per revolution: $\delta_1 = \arcsin s_e$ and $\delta_2 = \pi - \delta_1$. Linearizing the (δ,ω) system, the Jacobian at an equilibrium is

$$J = \begin{bmatrix} 0 & -1 \\ \kappa/I & -c/I \end{bmatrix}, \qquad \kappa = \tau_{max}\cos\delta_{eq}, \qquad \lambda^2 + \tfrac{c}{I}\lambda + \tfrac{\kappa}{I} = 0.$$

- $\delta_1 < \pi/2$: $\kappa > 0 \rightarrow$ **stable focus** (both eigenvalues in the left half-plane; complex because damping is weak). The operating point.
- $\delta_2 > \pi/2$: $\kappa < 0 \rightarrow$ **saddle**. Its stable manifold (the separatrix) bounds the basin of attraction; trajectories crossing it slip a pole (δ advances by 2π). This is the geometric picture of step-out.

The GUI starts the simulation *at* the stable equilibrium: $\delta(0) = \arcsin(1/M), \omega(0) = 2\pi f_0$ (`deltaInitial` / `stateAtStart` in `runSimulation`), so all observed transients are caused by the commanded maneuver.

5.4 Step-out ceiling

Lock requires $s_e \leq 1$:

$$\boxed{f_f \leq f_{max} = f_0\sqrt{M}} \qquad (M{=}5,\ f_0{=}160\text{ Hz} \Rightarrow f_{max} \approx 358\text{ Hz}).$$

The margin is *not* constant — it erodes quadratically with commanded frequency: $M_{eff}(f_f) = M(f_0/f_f)^2$.

5.5 Small-signal behavior: natural frequency and damping

Perturbing $\delta = \delta_{eq} + \varepsilon$ about the stable focus:

$$I\ddot{\varepsilon} + c\,\dot{\varepsilon} + \kappa\,\varepsilon = 0, \qquad \omega_n = \sqrt{\frac{\tau_{max}\cos\delta_{eq}}{I}}, \qquad \zeta = \frac{c}{2\sqrt{I\,\tau_{max}\cos\delta_{eq}}}.$$

With the file's numbers ($M = 5$, $f_0 = 160$ Hz $\to \tau_{max} = 5.00 \times 10^{-5}$ N·m, $\delta_{eq} = 11.5°$):

$$\omega_n \approx 112\text{ rad/s } (\approx 17.8\text{ Hz}), \qquad \zeta \approx 0.023\ (Q \approx 22), \qquad t_{settle} \approx \frac{4}{\zeta\omega_n} \approx 1.6\text{ s}.$$

Interpretation: the phase lock is a lightly-damped 18 Hz oscillator. Aerodynamic drag provides almost no phase damping, so every ramp ending rings for ~20 cycles. This is why the solver needs `MaxStep ≤ 2e−4 s` and why Hold segments need $\gtrsim 1.6$ s to settle within tolerance.

5.6 Ramp feasibility (quasi-static torque budget)

Perfect tracking of a ramp requires the drive-side torque to fit under τ_{max}:

$$2\pi I\,\dot{f}_f + k_d f_f^2 \leq \tau_{max} \quad \Longrightarrow \quad \boxed{\dot{f}_{max}(f_f) = \frac{\tau_{max} - k_d f_f^2}{2\pi I}}$$

At $f_0 = 160$ Hz, $M = 5$: $\dot{f}_{max} \approx 1.6$ kHz/s — far above the ~100 Hz/s ramps in the example schedule. The *practical* limit is usually dynamic, not quasi-static: an abrupt ramp end deposits energy into the $Q \approx 22$ phase oscillation, and if the swing peak carries δ across the separatrix (§5.3) the robot steps out below $\dot{f}_{max}$. Smooth blends (higher-order polynomial, exponential segments) exist precisely to taper $\dot{f}_f$ at boundaries.

---

6. Vertical (heave) dynamics

6.1 Blade-element thrust

The same element as §4.2 produces lift perpendicular to the local flow:

$$dL(r) = \tfrac{1}{2}\rho\, C_L\, c(r)\, v(r)^2\, dr \quad\Longrightarrow\quad T = \tfrac{1}{2}\rho C_L (2\pi f)^2 \int c(r)\, r^2\, dr \;\equiv\; k_T f^2.$$

Note thrust integrates $r^2 c\, dr$ (force) while drag torque integrates $r^3 c\, dr$ (force × arm). Both scale as f^2: **thrust and drag torque are inseparable**, with a geometry-fixed ratio $T/\tau_{drag} = (C_L/C_D)\,(\int cr^2)/(\int cr^3)$, independent of f. Rotor power follows: $P = \tau_{drag}\,\omega \propto f^3$.

6.2 Vertical equation of motion and hover

Newton in z (up positive):

$$m_R \ddot{z} = k_T f^2 - m_R\, g - D_z(\dot{z}).$$

Hover ($\ddot{z} = \dot{z} = 0$):

$$\boxed{f_h = \sqrt{\frac{m_R\, g}{k_T}}} \qquad\Longleftrightarrow\qquad k_T = \frac{m_R\, g}{f_h^2}\ \text{(calibration from a hover test).}$$

6.3 Heave damping from inflow

Climbing at $\dot{z}$ adds a vertical component to the relative wind at each blade element, tilting the inflow angle by $\Delta\phi \approx \dot{z}/(2\pi f r)$ and reducing the angle of attack $\alpha = \theta_{blade} - \phi$. With lift slope $a = dC_L/d\alpha$:

$$\Delta(dL) = \tfrac{1}{2}\rho\, a\, c(r)\, v^2 \cdot \Big(-\frac{\dot{z}}{2\pi f r}\Big) dr = -\tfrac{1}{2}\rho\, a\, c(r)\,(2\pi f)\, r\, \dot{z}\, dr,$$

$$\frac{\partial T}{\partial \dot{z}} = -\tfrac{1}{2}\rho\, a\,(2\pi f) \int c(r)\, r\, dr \;\equiv\; -k_w f \;<\; 0.$$

So the heave dynamics acquire **natural aerodynamic damping proportional to spin frequency** — spin rotors get altitude-rate stability for free:

$$\boxed{m_R \ddot{z} = k_T f^2 - m_R g - k_w f\, \dot{z}}$$

(The same inflow effect perturbs blade drag, making k_d weakly climb-dependent — the price of assumption A6.)

6.4 Linearization about hover: frequency → altitude

Let $f = f_h + \Delta f$, keep first order:

$$\Delta T = 2k_T f_h\, \Delta f = \frac{2m_R g}{f_h}\,\Delta f \quad\Longrightarrow\quad \ddot{z} = \frac{2g}{f_h}\,\Delta f - \frac{k_w f_h}{m_R}\,\dot{z}.$$

The frequency command is the altitude control input with gain $2g/f_h$. At $f_h = 160$ Hz: $2g/f_h \approx 0.12$ (m/s^2) per Hz. The example schedule's ±20 Hz steps command ≈ ±2.5 m/s² (a quarter-g). As a transfer function, with heave time constant $\tau_h = m_R/(k_w f_h)$:

$$\frac{\Delta z(s)}{\Delta f(s)} = \frac{2g/f_h}{s\,(s + 1/\tau_h)}.$$

---

7. The coupled nonlinear model

Stack the rotational and vertical dynamics. Thrust responds to the robot's **actual** spin frequency $\omega/2\pi$ (not the command) — the physically correct coupling:

State $x = [\delta,\ \omega,\ z,\ w]^\top$, **input** $u = f_f(t)$ (Hz):

$$\boxed{\begin{aligned}
\dot{\delta} &= 2\pi u - \omega \\
\dot{\omega} &= \frac{1}{I}\left[\tau_{max}\sin\delta - \frac{k_d}{4\pi^2}\,\omega|\omega|\right] \\
\dot{z} &= w \\
\dot{w} &= \frac{1}{m_R}\left[\frac{k_T}{4\pi^2}\,\omega^2 - m_R g - \frac{k_w}{2\pi}\,\omega\,w\right]
\end{aligned}}$$

Cascade structure. (δ, ω) evolve independently of (z, w); the vertical subsystem is driven by ω alone. The timescales stack:

$$\underbrace{\text{phase lock}}_{\sim 18\text{ Hz},\ \zeta\approx 0.02} \gg \underbrace{\text{rotor speed ramps}}_{\sim 1\text{-}10\text{ Hz}} \gg \underbrace{\text{heave}}_{\sim 0.1\text{-}1\text{ Hz}}$$

This separation legitimizes designing the layers independently — the altitude planner treats f as directly commandable *provided* it respects the inner loop's limits ($f \le f_0\sqrt{M}$, $\dot{f} \le \dot{f}_{max}$). The MATLAB GUI simulates exactly the inner two states of this model.

---

8. Linearized state-space model

8.1 Trim point

Hover at frequency f_h with the field commanding $u^* = f_h$:

$$x^* = \left[\ \delta^* = \arcsin\frac{k_d f_h^2}{\tau_{max}},\quad \omega^* = 2\pi f_h,\quad z^* = \text{any},\quad w^* = 0\ \right]^\top.$$

8.2 Jacobians

Define perturbations $\tilde{x} = x - x^*$, $\tilde{u} = u - u^*$, and the shorthand

$$\kappa = \tau_{max} \cos\delta^* \text{ (phase stiffness)}, \qquad c = \frac{k_d f_h}{\pi} \text{ (phase damping)}, \qquad \tau_h = \frac{m_R}{k_w f_h} \text{ (heave lag)}.$$

Differentiating the nonlinear model term by term (using $\partial_\omega[k_d\,\omega^2/4\pi^2] = k_d f_h/\pi = c$ and $\partial_\omega[k_T\,\omega^2/4\pi^2] = k_T f_h/\pi = m_R g/(\pi f_h)$ via $k_T f_h^2 = m_R g$):

$$\boxed{\dot{\tilde{x}} = A\,\tilde{x} + B\,\tilde{u}, \qquad y = C\,\tilde{x}}$$

$$A = \begin{bmatrix} 0 & -1 & 0 & 0 \\ \dfrac{\kappa}{I} & -\dfrac{c}{I} & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \dfrac{g}{\pi f_h} & 0 & -\dfrac{1}{\tau_h} \end{bmatrix}, \qquad B = \begin{bmatrix} 2\pi \\ 0 \\ 0 \\ 0 \end{bmatrix}, \qquad C = \begin{bmatrix} 0 & \frac{1}{2\pi} & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

with outputs $y = [\,f_{robot},\ z\,]^\top$ and $D = 0$. The lower-left block entry $g/(\pi f_h)$ is the thrust sensitivity to actual spin speed; the zero column 3 reflects that nothing depends on absolute altitude.

8.3 Eigenvalues (modes)

The block-triangular structure factors the characteristic polynomial:

$$\det(sI - A) = \underbrace{\left(s^2 + \tfrac{c}{I}s + \tfrac{\kappa}{I}\right)}_{\text{phase-lock mode}} \cdot \underbrace{\left(s + \tfrac{1}{\tau_h}\right)}_{\text{heave mode}} \cdot \underbrace{s}_{\text{altitude integrator}}$$

$$\lambda_{1,2} = -\zeta\omega_n \pm j\,\omega_n\sqrt{1-\zeta^2} \approx -2.6 \pm 112\,j\ \mathrm{s}^{-1}, \qquad \lambda_3 = -\frac{1}{\tau_h}, \qquad \lambda_4 = 0.$$

- $\lambda_{1,2}$: the 17.8 Hz, $\zeta \approx 0.023$ phase ringing (visible in every GUI run).
- λ_3: climb-rate settling, aerodynamic (inflow) damping.
- λ_4: altitude is an integrator — no feedback, marginally stable, as expected for open-loop height.

8.4 Input–output transfer functions

Command → robot frequency (from rows 1–2):

$$\frac{\tilde{f}_{robot}(s)}{\tilde{u}(s)} = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \qquad \text{(DC gain 1 : the robot tracks the command exactly at steady state)}.$$

Command → altitude (full cascade):

$$\frac{\tilde{z}(s)}{\tilde{u}(s)} = \frac{2g}{f_h} \cdot \frac{\omega_n^2}{(s^2 + 2\zeta\omega_n s + \omega_n^2)(s + 1/\tau_h)\,s}.$$

Steady climb rate per Hz of frequency offset: $w_{ss}/\Delta f = (2g/f_h)\,\tau_h = 2m_R g/(k_w f_h^2)$.

8.5 Structural properties

- **Controllability:** single input u enters only $\dot{\delta}$, but the chain $u \to \delta \to \omega \to w \to z$ makes (A, B) controllable for $\kappa, g, f_h \neq 0$ (generic hover). The robot is *underactuated* — one input, four states — but the cascade renders altitude controllable through frequency modulation.
- **Observability:** with $y = [f_{robot}, z]$, all four states are observable (δ is reconstructed from $\dot{f}_{robot}$ via the second state equation).
- **Validity boundary:** the linear model holds while (i) δ stays well inside the separatrix (no step-out: $f \leq f_0\sqrt{M}$, $\dot{f} \leq \dot{f}_{max}$),
  (ii) magnetics stay linear (below core saturation, drive near resonance so τ_{max} is frequency-flat), and (iii) climb rates are small enough that the hover-referenced k_d, k_T apply.

---

9. Parameter summary (values from the code / defaults)

Parameter	Value	Source
I	3.912×10^{-9} kg·m²	§3, four blades (code lines 21–24 use factor 2 → 3.901×10^{-9}; see §3 note)
k_d	3.91×10^{-10} N·m/Hz²	least-squares fit, lines 26–54
M (default)	5	GUI “Torque margin”
f_0 (example)	160 Hz	example schedule
τ_{max}	5.00×10^{-5} N·m	$Mk_d f_0^2$
δ_{eq}	11.5°	arcsin$(1/M)$
ω_n	112 rad/s (17.8 Hz)	§5.5
ζ	0.023	§5.5
t_{settle}	≈ 1.6 s	$4/\zeta\omega_n$
f_{max}	≈ 358 Hz	$f_0\sqrt{M}$
$\dot{f}_{max}(f_0)$	≈ 1.6 kHz/s	§5.6
$2g/f_h$	≈ 0.12 (m/s²)/Hz at f_h=160 Hz	§6.4
$m_R,\ k_T,\ k_w$	symbolic — calibrate via hover test ($k_T = m_R g/f_h^2$)	§6.2

---

10. Limitations and extensions

1. **Rotation-only attitude.** A1 removes tilt dynamics. Steering (tilting the field’s rotation plane) and righting maneuvers (e.g. inverted takeoff) need full rigid-body attitude dynamics: Euler’s equations + 3-D $\mathbf{m} \times \mathbf{B}$, where θ_f, θ_r are no longer scalars.
2. **Constant τ_{max}.** Real drives (resonant LC coil drive, cored coils) have frequency- and amplitude-dependent field: replace $\tau_{max} \to \tau_{max}(f_f)$ from a measured current-vs-frequency sweep.
3. **Hover-referenced aerodynamics.** Aggressive climb changes inflow → $k_d(f, \dot{z})$, $k_T(f, \dot{z})$ from momentum theory; the swing equation’s drag load then couples to the vertical state, breaking the one-way cascade weakly.
4. **Torque ripple.** Field harmonics (core saturation flat-topping, non-ideal coil phasing) inject ripple at $2f, 4f, \ldots$; components landing near ω_n are amplified $Q \approx 22$ times.

5. **No lateral dynamics.** Position in the plane, precession-based translation, and wall/ground aerodynamic effects are all outside this model.

Appendix A — Correspondence with the MATLAB implementation

Model element	Code location (search anchor)
I assembly (§3, four blades)	`%% Fixed robot model` block: `I_body`, `n_blades`, `I_blade`, `I_robot`
drag data + through-origin LS fit + R^2	`fre_points`, `drag_torque_points`, `k_drag = sum(...)`, `R_squared`
$\tau_{max} = Mk_d f_0^2$	`tauMagMax = torqueMargin * tauRequiredInitial`
initial condition $[\arcsin(1/M),\ 2\pi f_0]$	`deltaInitial = asin(...)`, `stateAtStart`
two-state ODE	`rotationODE` in `runSimulation`
per-segment `ode45`, state handoff	segment loop in `runSimulation` (`MaxStep` resolves the ω_n ringing)
segment shapes: hold / s^n / normalized exponential blend	`evaluateSegmentFrequency`
wrapped δ, tracking error, pole-slip count	`deltaWrapped`, `frequencyError`, `netPhaseTurns`
Hold-tail settling check	`buildHoldSummary`

The GUI integrates states 1–2 of the §7 model with $u = f_f(t)$ built from piecewise segments; states 3–4 (vertical) are the analytical extension derived in §6.

Appendix B — Why $\dot{f}_{max}$ and f_{max} are the two feasibility axes

The torque budget $\tau_{max} \geq 2\pi I\dot{f}_f + k_d f_f^2$ is a single inequality in two variables $(f_f, \dot{f}_f)$: its boundary is a downward parabola in the $(f_f^2, \dot{f}_f)$ plane. $f_{max} = f_0\sqrt{M}$ is its $\dot{f}_f = 0$ intercept; $\dot{f}_{max}(f)$ is a horizontal slice. Any commanded trajectory must remain inside this region *with dynamic margin* for the $Q \approx 22$ phase swing excited at segment boundaries — a conservative rule of thumb is to keep the quasi-static demand below $\tau_{max} \sin(\pi/2 - \Delta)$ with a swing allowance Δ of a few tens of degrees, or to shape $\dot{f}_f$ continuously (higher-order polynomial / exponential segments) so little swing energy is injected.